Design Document

Author	*Personal Code*
Nome 1	Matricola 1
Nome 2	Matricola 2
Nome 3	Matricola 3

Settembre, 2025

Contents

1. Introduction

1.1 Purpose

Triplo is a cross-platform mobile application for smartphones and smartwatches, conceived as a digital logbook for trekking activities.
The application aims to enhance the experience of hiking in mountain environments by combining route information, environmental data, and recording of personal activity.
Users can explore predefined trekking routes and access useful information about the paths, including navigation support and contextual data such as weather conditions.
At the same time, the application allows hikers to document their experiences by creating logbook entries enriched with photos and personal diary notes.
The platform also enables users to discover and follow other hikers, and to browse selected entries from their trekking diaries, enhancing the sharing of experiences and inspiration for future hikes.

During a hike, users can optionally complete challenges, such as time-based goals designed to enrich the trekking experience.
The following sections describe the system architecture, the main functionalities of the application, and the technological choices adopted in its development.

1.2 Scope

Triplo supports users during trekking activities by providing guidance, contextual information, and tools to document their experiences in mountain environments.
The application focuses on the Madonna di Campiglio area and offers a set of predefined trekking routes with different levels of difficulty.
During a hike, users can access information related to the selected route, including navigation support and environmental conditions that may influence the trekking experience.
The application also encourages engagement through optional challenges that users may complete along the route. Each trekking activity can be recorded in a personal digital logbook, where users can document their experience by adding photos and diary notes. Users can also browse selected entries from the logbooks of other hikers they follow, allowing them to discover new routes and gain inspiration from shared experiences.
The application aims to support outdoor exploration while enabling users to record, revisit, and share meaningful moments from their trekking activities.

1.3 Definitions, Acronyms, Abbreviations

- LTE (Long Term Evolution) is a mobile communication standard. LTE is also referred to as "4G", the fourth generation of mobile communication technology. LTE consists of a number of enhancements to the "Universal Mobile Telecommunications System" (UMTS) – the third generation of mobile communications.

1.4 References

1.4.1 Main References

- Design and Implementation of Mobile Applications Lectures and Slides
-
- OpenWeather API Documentation
  https://openweathermap.org/api/one-call-3?collection=one_call_api_3.0
- What is LTE?
  https://www.telekom.com/en/company/details/the-nine-most-important-facts-about-lte-606
- Severe Weather Alerts API
  https://www.weatherbit.io/api/alerts
- What Is Model-View-Controller (MVC)?
  https://www.oracle.com/technical-resources/articles/javase/application-design-with-mvc
  html

2. Architectural Design

2.1 Overview

2.1.1 High-level components

The Triplo system is designed according to a modular architecture in which the main application logic is implemented on the client side, while backend functionalities are delegated to managed services and external APIs, as illustrated in Figures 2.1 and 2.2.

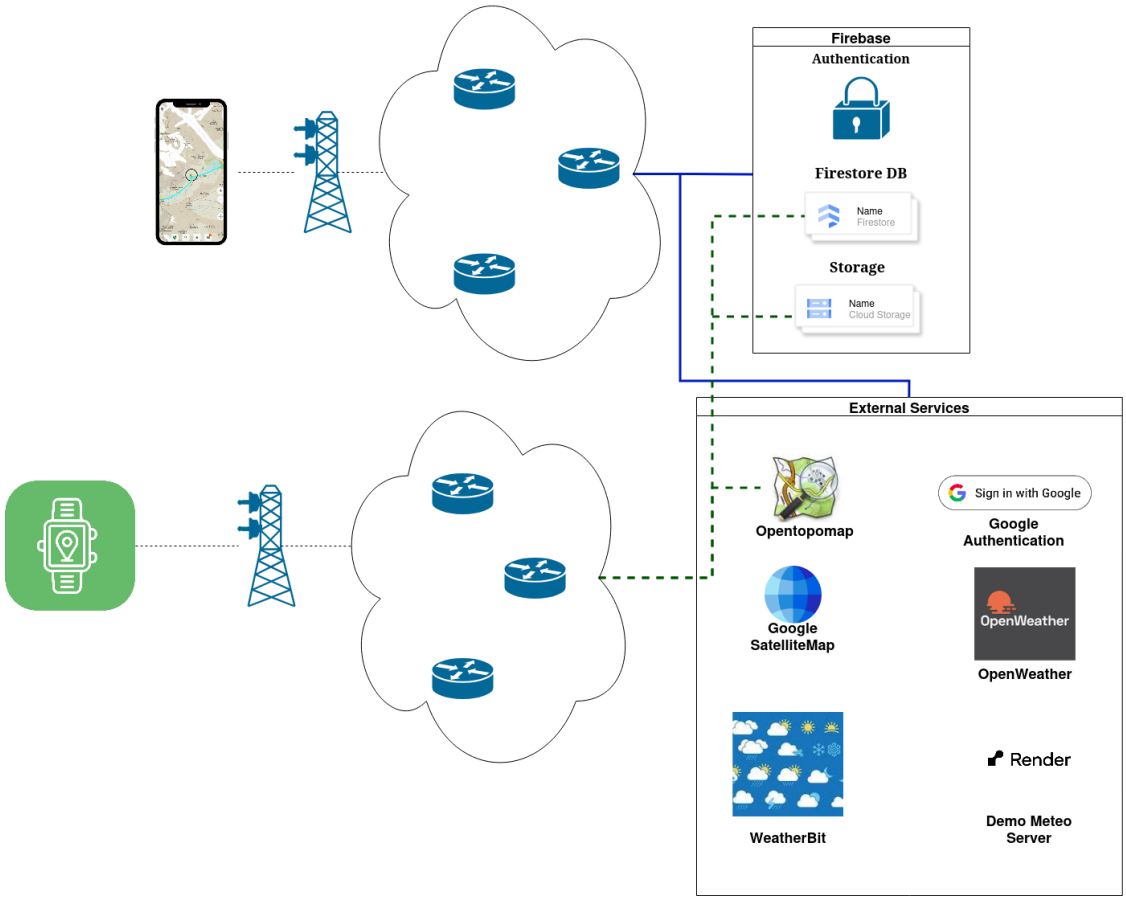

Figure 2.1: Overview Diagram: Blue lines denote the smartphone application's connectivity with Firebase and external services, providing full access to all Firebase modules and external APIs. Dashed green lines denote the smartwatch application's connectivity, which is limited to selected Firebase services, Firestore and Storage and specific external services, for example OpenTopoMap.

The system separates concerns between presentation, application logic, and data management. Both the presentation layer and the core application logic are implemented within the Flutter-based applications. In particular, the mobile application represents the main client, while the smartwatch application provides a lightweight extension offering a subset of functionalities adapted to the constraints and interaction model of wearable devices, relying on the same backend services and data sources.

The application directly manages user interactions, business logic, and coordination between components.
Data management and authentication are handled through Firebase services, specifically Firebase Authentication, Cloud Firestore, and Firebase Storage. The system leverages Firebase as a Backend-as-a-Service (BaaS) platform, providing managed services for authentication, data storage and user management, relying on platform-specific SDKs for access to these resources. This design choice shifts part of the traditional server-side responsibilities to the client, which becomes responsible for orchestrating application logic and interactions with backend services.
The adoption of Backend-as-a-Service (BaaS) solutions enables the system to replace a traditional custom server-side layer with a more lightweight and scalable architecture. By delegating data management, authentication, and storage to managed services, architectural complexity is reduced by eliminating the need for maintaining a dedicated custom backend infrastructure. In addition to Firebase, the system integrates external third-party services (e.g., weather, map, and satellite APIs), which are accessed via RESTful APIs over HTTPS.
This architectural approach emphasizes a client-driven design, reduces backend complexity, and enables scalability by leveraging managed cloud services.

2.1.2 High level components interaction

The mobile application represents the entry point of the system and runs on the user's device. It is responsible for handling user interaction, application logic, and coordination between components according to an MVC-inspired architecture.
The system relies on different application components to interact with the required services.
In particular, domain controllers, for example the `UserController`, `DiaryController`, `TrekkingController`, `ChallengeController` coordinate access to data and backend resources, while specialized components such as the `APIController` manage communication with external third-party APIs.
Data persistence, authentication, and file storage are handled through Firebase, which is adopted as a Backend-as-a-Service (BaaS) platform. Interaction with Firebase services (Authentication, Cloud Firestore, and Storage) is performed through platform-specific SDKs rather than through explicitly defined REST APIs.
Communication with external third-party services (e.g., weather, map, or satellite APIs) is instead performed using RESTful APIs over HTTPS.
This approach removes the need for a dedicated backend gateway or custom server-side components, allowing the application to directly orchestrate interactions with backend services and external resources.

2.2 Component view

2.2.1 High level component view

2.2.2 Mobile Smartphone App component view

Domain controllers, such as `UserController`, `DiaryController`, `TrekkingController`, and `ChallengeContro` are responsible for coordinating the application logic related to their respective functional domains. These controllers manage interactions between the View layer and the Model layer, and they also mediate access to backend resources such as Firebase services.

The `APIController`, instead, has a more specialized role. It is not intended as a gateway to Firebase, but rather as a component dedicated to managing the communication with external third-party services, such as weather, map, satellite, or other external APIs required by the application.

Therefore, in the proposed architecture:

- domain controllers manage the application use cases and the interaction with Firebase-related resources;
- `APIController` manages the communication with external non-Firebase services, it is specifically responsible for handling communication with external third-party APIs (e.g., weather, map, or satellite services), which are conceptually distinct from Firebase-managed services. For this reason, the name `APIController` is preserved for consistency with the existing codebase, while its responsibilities are clearly defined within the architectural description.
- Firebase is modeled as a set of backend services used by the application, rather than as a custom backend.

2.2.3 Mobile Smartwatch App component view

2.3 Deployment view

2.4 Runtime view

2.4.1 Selected Use Cases

The identified use cases provide an abstract representation of the interactions between the user and the system. They have been selected to capture the essential functionalities of the application and to model its observable behavior from the user's perspective.
These set of use cases is representative of the main system capabilities and supports a clear understanding of the services offered to the user.

- Add Diary Page
- Explore trekkings on Map
- View trekking information
- View weather information
- Search Users
- Search trekking
- View compass and altitude information
- Participate in Route Challenge
- Login
- Register
- Pair Smartwatch
- Update User Profile

2.4.2 Add Diary Page use case sequence diagram

The most important part of the *Add Diary Page* sequence diagram is the save phase, where the data entered by the user is stored in the system.

When the user presses the save button, the `DiaryPage` starts the process by showing a loading page and then it formats the date, computing the duration from hours and minutes. If the user has selected photos, the page calls the `DiaryController` to upload them to `FirebaseStorage`. The controller uploads each image and returns a list of paths, which will be stored in the diary instance.

After this step, the `DiaryPage` calls the `addDiary(...)` method of the `DiaryController`, passing all the collected data. The controller retrieves the current user identifier from `FirebaseAuth`, generates a new unique identifier for the diary, and creates a `DiaryModel` object.

Before saving the diary in `FirebaseFirestore`, the object is converted into a `Map<String, dynamic>`. This is necessary because Firestore stores data in a JSON-like format, which is based on key-value pairs. The map represents the diary as a set of fields (such as date, duration, notes, etc.), where each field is associated with a value. The type `dynamic` is used because these values can have different types, such as strings, numbers, booleans, or lists.

Once converted, the map is stored in the `diary` collection in Firestore. At the same time, the controller updates the user document, adding the diary identifier to either the public or private diary list, depending on the selected visibility.

Finally, the control returns to the `DiaryPage`, which updates the user level through the `UserController` and navigates to the `UserPage`, completing the operation.

2.4.3 Watch–Phone Pairing Mechanism

Overview

The pairing mechanism associates a Wear OS device with an authenticated mobile user through a backend-mediated approval process. The watch does not authenticate directly with Firebase Auth; instead, device identity and user identity are linked via Firestore.

Device Identity

Each watch generates (or restores) a persistent `watchId`, stored securely on-device. A corresponding document exists in:

```
watch/{watchId}
```

The `watchId` uniquely identifies the physical device and is never deleted during logout.

Pairing Flow

1. Initialization (Watch) The watch generates a random UUID (`qrToken`) and writes:

```
watch_pair/{watchId}
```

with:

- `status = "waiting"`
- `uid = null`
- `qrToken`
- `createdAt`, `expiresAt`

A QR code encoding `watchId` and `qrToken` is displayed.

2. Approval (Phone) After scanning the QR, the authenticated mobile app:

- Verifies token validity and expiration.
- Updates the document with:

- `status = "approved"`
- `uid = request.auth.uid`

3. Completion (Watch) The watch listens to the document in real time. When `status == "approved"` and `uid` is set, pairing is considered complete and the watch loads user data using that `uid`.

Session Restoration

On startup, the watch checks:

`watch_pair/{watchId}`

If the document is already approved, the session is restored automatically.

2.4.4 Logout

Logout does not delete the device identity. Instead, the watch resets:

- `status = "waiting"`
- `uid = null`

This preserves device identity while removing user association.

2.4.5 Security Properties

- Only the watch may create/reset pairing in `waiting` state.
- Only authenticated users may approve pairing.
- QR tokens are random and time-limited.

This design cleanly separates device identity from user authentication while minimizing credential exposure on the wearable device.

2.5 Component interfaces

2.6 Selected architectural styles and patterns

2.7 Other Design Decisions

2.7.1 Network Connectivity Requirements and Platform Justification

The smartwatch application requires reliable access to internet connectivity in order to provide a complete and consistent user experience. In particular, network access is necessary for several core functionalities of the system.

The application relies on external services to retrieve real-time data, such as maps, which are essential for trekking activities.

Internet access enables communication with backend services and APIs, allowing the application to fetch dynamic content and maintain updated information.

Given these requirements, the choice of Wear OS as the target platform is appropriate and justified. Wear OS supports multiple network communication modes, including:

- **Bluetooth connectivity**: allows the smartwatch to access the internet indirectly through a paired smartphone.
- **Wi-Fi connectivity**: enables the device to connect directly to wireless networks, allowing standalone operation in specific scenarios.

- **Cellular connectivity (LTE with eSIM)**: available on selected devices, providing full independence from the smartphone and ensuring continuous connectivity during outdoor activities.

This flexibility ensures that the application can operate under different conditions, ranging from smartphone-assisted usage to fully autonomous scenarios. In particular, the availability of LTE-enabled devices makes Wear OS especially suitable for outdoor and trekking contexts, where the user may not always have access to their smartphone.